

Supplementary Code: Random Forest Regression Model for SO₂ Concentration Prediction

This document reproduces, in full, the Python script (RF.py) used to train and evaluate the Random Forest regression model reported in the manuscript. The model predicts the SO₂ concentration from four input features—relative humidity, absorbent weight, temperature, and time. The workflow comprises: (i) loading the experimental data, (ii) min–max normalisation of the features and target, (iii) a 70/20/10 train/validation/test split, (iv) training a tuned Random Forest regressor, (v) computing the standard error metrics (MSE, RMSE, R^2 , MAE), and (vi) generating a cross-validated learning curve. The code is reproduced verbatim below; line numbers are provided for ease of reference.

Software: Python 3 with pandas, numpy, scikit-learn, and matplotlib.

```

1  # ===== Imports =====
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6  from sklearn.ensemble import RandomForestRegressor
7  from sklearn.model_selection import train_test_split, learning_curve
8  from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
9  from sklearn.preprocessing import MinMaxScaler
10
11 # ===== Data Loading =====
12 excel_file = 'data.xlsx'
13 sheet_name = 'Sheet1'
14 data_kami2 = pd.read_excel(excel_file, sheet_name=sheet_name)
15
16 new_column_names = ['Relative humidity', 'Absorbent weight', 'Temperature', 'Time',
17                     r'$SO_2$ concentration']
18 data_kami2.columns = new_column_names
19
20 # ===== Preprocessing =====
21 Xf = data_kami2.iloc[:, 0:4]
22 yf = data_kami2[[r'$SO_2$ concentration']]
23
24 X = Xf.values
25 Y = yf.values
26
27 scaler_X = MinMaxScaler()
28 X_normalized = scaler_X.fit_transform(X)
29
30 scaler_y = MinMaxScaler()
31 Y_normalized = scaler_y.fit_transform(Y)
32
33 # ===== Train / Val / Test Split (70/20/10) =====
34 X_train_split, X_temp, Y_train_split, Y_temp = train_test_split(
35     X_normalized, Y_normalized, test_size=0.3, random_state=20, shuffle=True

```

```

36 X_test_split, X_val_split, Y_test_split, Y_val_split = train_test_split(
37     X_temp, Y_temp, test_size=2/3, random_state=20, shuffle=True
38 )
39
40 X_train = X_train_split
41 Y_train = Y_train_split
42 X_val = X_val_split
43 Y_val = Y_val_split
44 X_test = X_test_split
45 Y_test = Y_test_split
46
47 # ===== Model Definition & Training =====
48 best_rf = RandomForestRegressor(
49     n_estimators=9,
50     max_depth=16,
51     min_samples_leaf=1,
52     criterion='poisson',
53     min_samples_split=2,
54     n_jobs=-1
55 )
56 best_rf.fit(X_train, Y_train.ravel())
57
58 # ===== Evaluation =====
59 Y_pred_train = best_rf.predict(X_train)
60 Y_pred_val = best_rf.predict(X_val)
61
62 mse_train = mean_squared_error(Y_train.ravel(), Y_pred_train)
63 rmse_train = np.sqrt(mse_train)
64 r2_train = r2_score(Y_train.ravel(), Y_pred_train)
65 mae_train = mean_absolute_error(Y_train.ravel(), Y_pred_train)
66
67 mse_val = mean_squared_error(Y_val.ravel(), Y_pred_val)
68 rmse_val = np.sqrt(mse_val)
69 r2_val = r2_score(Y_val.ravel(), Y_pred_val)
70 mae_val = mean_absolute_error(Y_val.ravel(), Y_pred_val)
71
72 print("Evaluation Metrics for the Best Model:")
73 print(f"Training set: MSE = {mse_train:.4f}, RMSE = {rmse_train:.4f}, R2 = {r2_train:.4f}, MAE = {mae_train:.4f}")
74 print(f"Validation set: MSE = {mse_val:.4f}, RMSE = {rmse_val:.4f}, R2 = {r2_val:.4f}, MAE = {mae_val:.4f}")
75
76 # ===== Learning Curve =====
77 train_sizes, train_scores, val_scores = learning_curve(
78     estimator=best_rf,
79     X=X_train,
80     y=Y_train.ravel(),
81     cv=5,
82     scoring='neg_root_mean_squared_error',
83     train_sizes=np.linspace(0.1, 1.0, 10),
84     n_jobs=-1
85 )
86
87 train_rmse = -np.mean(train_scores, axis=1)
88 val_rmse = -np.mean(val_scores, axis=1)
89
90 plt.figure(figsize=(8, 6))
91 plt.plot(train_sizes, train_rmse, 'o-', label='Training RMSE')
92 plt.plot(train_sizes, val_rmse, 'o-', label='Validation RMSE')

```

```
93 plt.xlabel('Training Set Size')
94 plt.ylabel('RMSE')
95 plt.title('Learning Curve (Random Forest)')
96 plt.legend()
97 plt.grid(True)
98 plt.tight_layout()
99 plt.show()
```